CST-109: Cloud Security

Assignment 3

Design and Implementation of a Secure Cloud Application Architecture

Authors:

Utkarsh Kumar (23114101)
Adesh Kaduba Palkar (23114004)

Contents

1 Introduction **2**

2 Secure Cloud Architecture **2**
2.1 System Components 2
2.2 Transport Security 3
2.3 Distributed File Storage 3

3 Authentication and Authorization **3**
3.1 Token-Based Authentication 3
3.2 Role-Based Access Control 3

4 Threat Modeling Using STRIDE **4**

5 Automated Mitigation Engine **4**

6 Attack Simulation and Evaluation **5**
6.1 Brute Force Attack 5
6.2 Denial of Service 5
6.3 Token Tampering 5
6.4 Replay Attack 5
6.5 IDOR Attack 5

7 Logging and Monitoring **5**

8 Conclusion **6**

1 Introduction

Modern cloud applications are highly distributed and exposed to multiple security risks. Attackers often exploit vulnerabilities in authentication systems, network communication, or data storage components. Therefore, designing a secure cloud architecture requires strong encryption, secure authentication mechanisms, monitoring infrastructure, and automated threat mitigation.

This assignment implements a prototype secure cloud system with the following objectives:

- Provide secure TLS communication between all system components.
- Implement token-based authentication with IP binding and expiry control.
- Enforce role-based access control for different users.
- Perform STRIDE threat modeling to identify security risks.
- Implement automated mitigation techniques such as rate limiting and IP blocking.
- Simulate common attacks to evaluate resilience.

The architecture demonstrates how layered security mechanisms can protect a distributed cloud service.

2 Secure Cloud Architecture

2.1 System Components

The architecture consists of five main components:

Client Provides a command-line interface for interacting with the cloud service. Users can upload, download, list, and delete files. The client also contains built-in attack simulation functions.

API Gateway Serves as the single entry point for all external traffic. It performs request validation, rate limiting, and forwards legitimate requests to the application server.

Application Server Handles authentication, authorization, and business logic. It also coordinates storage operations across multiple storage nodes.

Storage Servers Four independent storage servers store file chunks and verify integrity using SHA-256 checksums.

Monitoring Module Aggregates log files from different components and provides security monitoring and reporting.

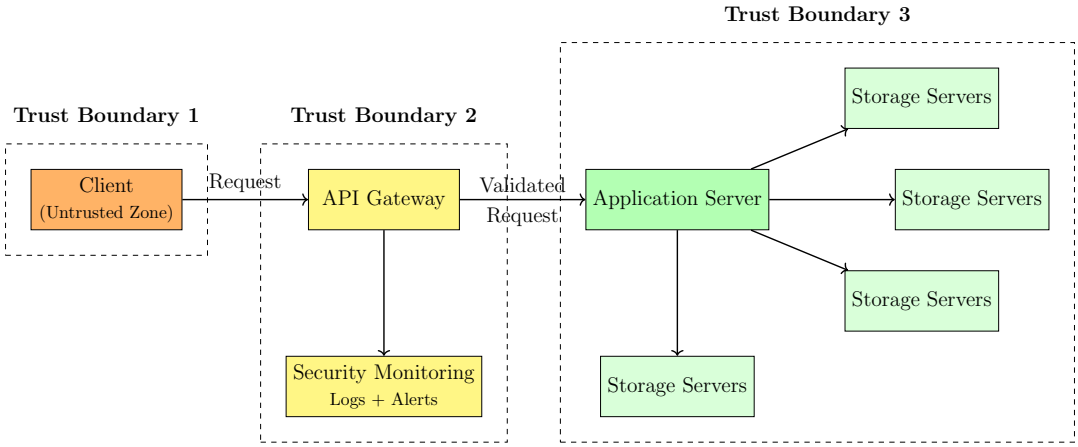

Figure 1: Secure Cloud Architecture with Trust Boundaries

2.2 Transport Security

All communication between components is secured using TLS encryption. Self-signed RSA certificates are generated using OpenSSL and loaded into the Python `ssl` module.

```
openssl req -x509 -newkey rsa:2048 -keyout server.key \
-out server.crt -days 365 -nodes
```

TLS ensures confidentiality and integrity of network communication and protects against man-in-the-middle attacks.

2.3 Distributed File Storage

Files uploaded by clients are divided into 1MB chunks. These chunks are distributed across storage servers in a round-robin manner. Each storage server computes a SHA-256 hash for the stored chunk and records it in a manifest file. During downloads, hashes are verified to detect tampering.

3 Authentication and Authorization

3.1 Token-Based Authentication

Authentication is implemented using cryptographically secure tokens generated using Python's `secrets` module.

```
token = secrets.token_urlsafe(32)
expires = datetime.now() + timedelta(hours=24)
self.tokens[token] = {"user":user, "ip":client_ip,
                      "expires":expires}
```

Each token is associated with a user, client IP address, and expiration timestamp. Requests are rejected if the token is invalid, expired, or used from a different IP address.

3.2 Role-Based Access Control

Three user roles are implemented:

Role	Upload	Download	Delete
Admin	Yes	Yes	Yes
User	Yes	Yes	No
ReadOnly	No	Yes	No

Access permissions are verified for every request before execution.

4 Threat Modeling Using STRIDE

The STRIDE methodology was used to identify security threats in the system. STRIDE categories include:

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege

Each threat is evaluated using a likelihood and impact score from 1 to 5.

Threat	Score	Risk Level
Credential spoofing	20	Critical
Payload tampering	20	Critical
Token manipulation	20	Critical
Denial of service	20	High
Unauthorized access	16	High
Data tampering	10	Medium

The threat modeling module also analyzes system log files to correlate threats with real events such as authentication failures and rate limit violations.

5 Automated Mitigation Engine

To defend against attacks automatically, the system includes a mitigation engine integrated with the API Gateway.

Key mitigation policies include:

- Rate limit: 30 requests per minute per IP
- Brute-force threshold: 5 failed login attempts
- Account lockout duration: 5 minutes

- DoS threshold: 100 requests per minute
- IP block duration: 10 minutes

```
now=time.time()
self.requests[ip]=[t for t in self.requests[ip] if now-t<60]
if len(self.requests[ip])>=RATE_LIMIT:
    reject_request()
```

These controls ensure malicious traffic is blocked before reaching core services.

6 Attack Simulation and Evaluation

Five attacks were simulated using the client attack tools.

6.1 Brute Force Attack

Repeated login attempts were issued against the admin account. After five failed attempts, the system locked the account and blocked further requests.

6.2 Denial of Service

Multiple concurrent threads generated high request volumes. Rate limiting successfully rejected requests beyond the configured threshold.

6.3 Token Tampering

Fabricated tokens and modified payloads were submitted. Since tokens are stored server-side, all invalid tokens were rejected.

6.4 Replay Attack

Expired tokens were reused after expiration. The application server rejected them during validation.

6.5 IDOR Attack

Unauthorized operations and path traversal attempts were tested. All requests violating permissions were blocked.

During testing the monitoring system recorded over 500 log entries and dozens of mitigation actions, confirming the effectiveness of the security mechanisms.

7 Logging and Monitoring

All components generate structured logs that include timestamp, module name, severity level, and message.

```
%(asctime)s - %(name)s - %(levelname)s - %(message)s
```

Logs are collected by the monitoring module which provides both report generation and live monitoring capabilities.

8 Conclusion

This project demonstrates a secure cloud architecture implementing encryption, authentication, authorization, monitoring, and automated threat mitigation. The STRIDE threat modeling process helped identify critical risks while attack simulations validated the effectiveness of implemented defenses.

Future improvements could include:

- JWT-based authentication for distributed scaling
- Strong password hashing using bcrypt or Argon2
- Mutual TLS authentication between services
- Integration with enterprise SIEM tools

References

[1] Microsoft Security Development Lifecycle Team, *The STRIDE Threat Model*. Microsoft, 2006.

[2] OWASP Foundation, *OWASP Top Ten 2021*. Open Web Application Security Project, 2021.

[3] Ross, R. et al., *NIST SP 800-53 Rev. 5 – Security and Privacy Controls for Information Systems and Organizations*. National Institute of Standards and Technology, 2020.

[4] Rescorla, E., *The Transport Layer Security (TLS) Protocol Version 1.3*. RFC 8446, IETF, 2018.

[5] Python Software Foundation, *ssl — TLS/SSL wrapper for socket objects*. Python 3 Standard Library Documentation, 2024.